



中国海洋大学
计算机科学与技术学院

《系统开发工具基础 • Week 2》

实验报告

姓名 潘星睿

学号 25020007096

指导教师 周小伟

日期 September 5, 2026

GitHub 仓库 <https://github.com/HiyamaHina/->



摘要

本次实验围绕 **Linux** 开发工具链的核心技能展开，共完成四个规定任务及十道课后练习题。

任务涵盖：**Shell** 后台任务控制与信号处理、**Python** 语言服务器下的代码导航与重构、调试器定位归并排序缺陷、以及性能分析优化词频统计程序。

关键词：Shell 任务控制；语言服务器；调试器；性能分析；Git 与 LaTeX 协作

本次试验 github 的 commit 记录如下：

 q05	q05 finished	last week
 q06	Q12,4th Sept.	yesterday
 q07	Q12,4th Sept.	yesterday
 q08	q08 finished	5 days ago
 1.pdf	report 1	last week
 1.tex	Q11	yesterday
 Q10.png	Q10	last week
 Q11.png	Q11	yesterday
 Q13.png	Q13	yesterday
 Q14.png	using --	2 hours ago
 Q1501.png	export and printev	1 hour ago
 Q1502.png	export and printev	1 hour ago
 Q17.png	sleep and wait	1 hour ago
 Q18.png	sort	23 minutes ago
 Q19.png	Q19	1 minute ago
 Q20.png	Q19	1 minute ago

Contents

摘要	1
1 第 5 题：控制一个可清理的后台任务	3
1.1 题目要求	3
1.2 实现过程	3
1.3 踩坑记录	3
1.4 验证结果	4
2 第 6 题：语义重构与本地开发反馈	5
2.1 题目要求	5
2.2 实现过程	5
2.3 踩坑记录	7
3 第 7 题：用调试器定位归并排序缺陷	7
3.1 题目要求	7
3.2 实现过程	7
3.3 踩坑记录	9
4 第 8 题：先测量，再优化慢速词频程序	9
4.1 题目要求	9
4.2 实现过程	9
5 课后题	11
5.1 第 1 题	11
5.2 第 2 题	12
5.3 第 3 题	13
5.4 第 4 题	14
5.5 第 5 题	15
5.6 第 6 题	16
5.7 第 7 题	17
5.8 第 8 题	18
5.9 第 9 题	19
5.10 第 10 题	19
个人感想	21
总结	21

1 第 5 题：控制一个可清理的后台任务

1.1 题目要求

在 q05 目录下完成以下操作：

1. 赋予脚本执行权限，在前台启动，并把 `stdout`、`stderr` 分别写入 `stdout.log`、`stderr.log`。
2. 使用 `Ctrl-Z` 挂起任务，再用 `bg` 让它在后台继续；使用 `jobs` 确认状态。
3. 使用 `jobs-p` 或等价方式取得 PID，不得手工抄写 PID；发送 `SIGTERM` 并等待任务结束。
4. 确认 `cleanup.log` 中出现 `CLEAN_EXIT`，且任务已经退出；禁止使用 `kill-9`。

1.2 实现过程

使用 Shell 命令完成上述任务。关键命令如下：

```
1 chmod +x worker.sh
2 ./worker.sh > stdout.log 2> stderr.log # 然后按 Ctrl-Z 挂起
3 bg
4 jobs
5 pid=$(jobs -p %1)
6 kill -TERM "$pid"
7 wait %1
8 cat cleanup.log
9 jobs
```

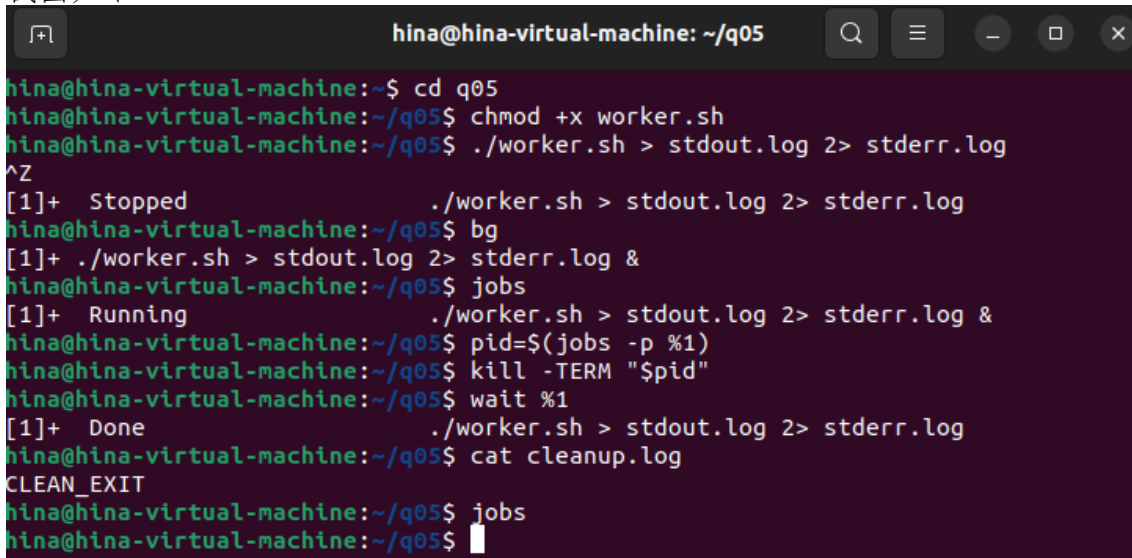
Listing 1. 代码

1.3 踩坑记录

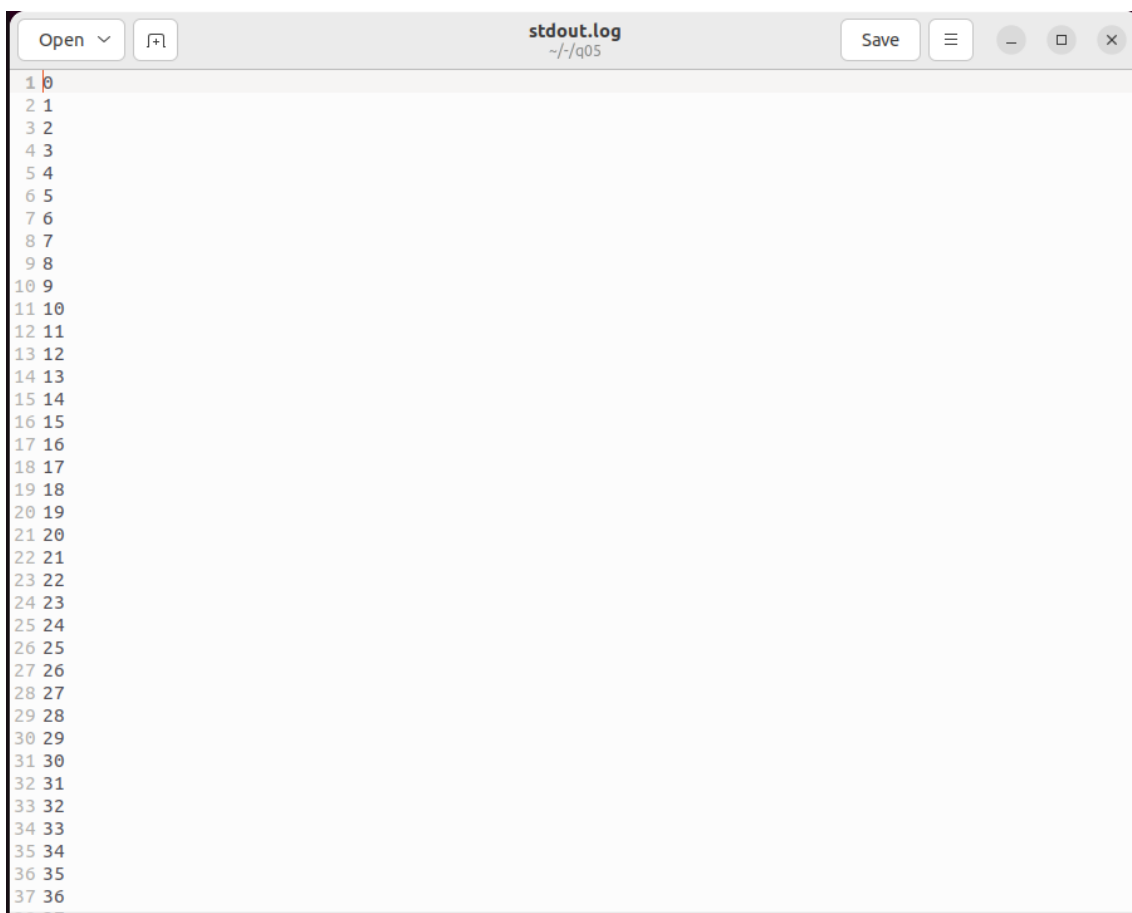
1. 第一次忘记 `bg`。

1.4 验证结果

截图如下



```
hina@hina-virtual-machine: ~/q05
hina@hina-virtual-machine:~$ cd q05
hina@hina-virtual-machine:~/q05$ chmod +x worker.sh
hina@hina-virtual-machine:~/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
hina@hina-virtual-machine:~/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
hina@hina-virtual-machine:~/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
hina@hina-virtual-machine:~/q05$ pid=$(jobs -p %1)
hina@hina-virtual-machine:~/q05$ kill -TERM "$pid"
hina@hina-virtual-machine:~/q05$ wait %1
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
hina@hina-virtual-machine:~/q05$ cat cleanup.log
CLEAN_EXIT
hina@hina-virtual-machine:~/q05$ jobs
hina@hina-virtual-machine:~/q05$
```



```
stdout.log
~/q05
1 0
2 1
3 2
4 3
5 4
6 5
7 6
8 7
9 8
10 9
11 10
12 11
13 12
14 13
15 14
16 15
17 16
18 17
19 18
20 19
21 20
22 21
23 22
24 23
25 24
26 25
27 26
28 27
29 28
30 29
31 30
32 31
33 32
34 33
35 34
36 35
37 36
```

2 第 6 题：语义重构与本地开发反馈

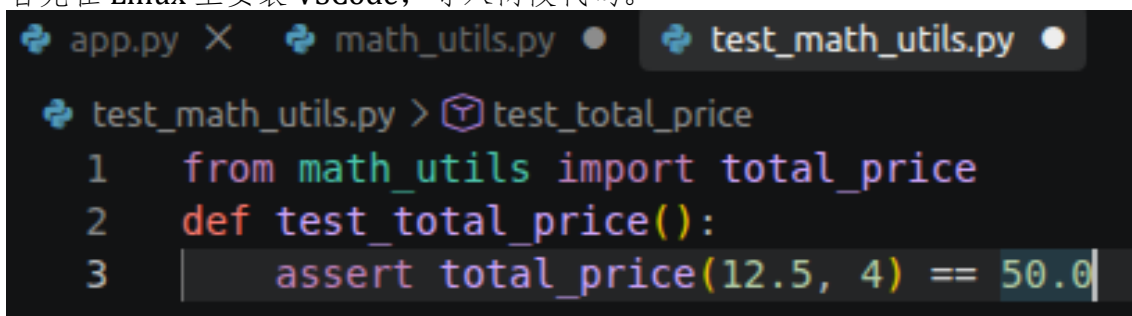
2.1 题目要求

在 q06 目录下完成以下操作：

1. 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
2. 分别演示跳转到定义和查找引用。
3. 使用“重命名符号”把 `total_price` 改为 `calculate_total`，保证定义和所有代码引用同步改变。
4. 在 `app.py` 中临时加入未使用的 `import os`，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

2.2 实现过程

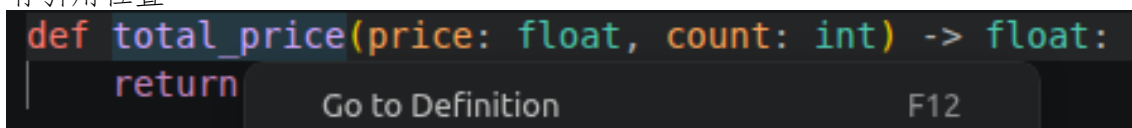
首先在 Linux 上安装 VSCode，导入两段代码。



```
app.py × math_utils.py test_math_utils.py
test_math_utils.py > test_total_price
1 from math_utils import total_price
2 def test_total_price():
3     assert total_price(12.5, 4) == 50.0
```

有语法高亮，确认语言服务器激活

将光标放在 `total_price` 上，按 F12 跳转到 `math_utils.py` 定义处；按 Shift+F12 查看所有引用位置



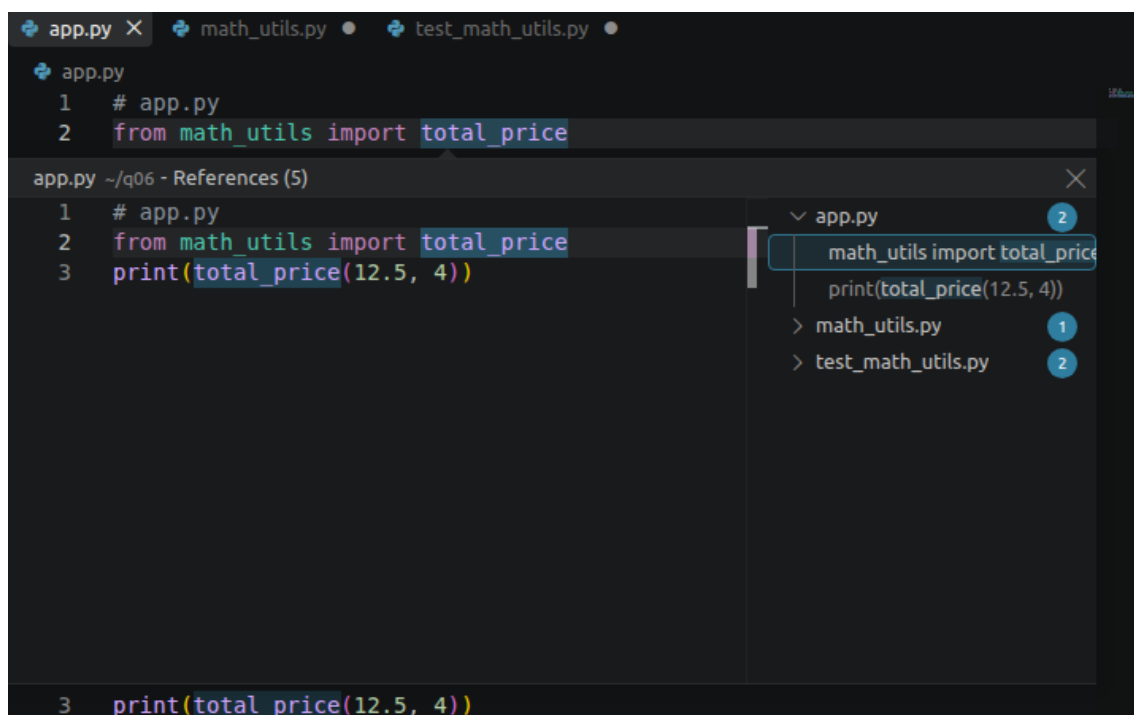
```
def total_price(price: float, count: int) -> float:
    return
```

Go to Definition F12

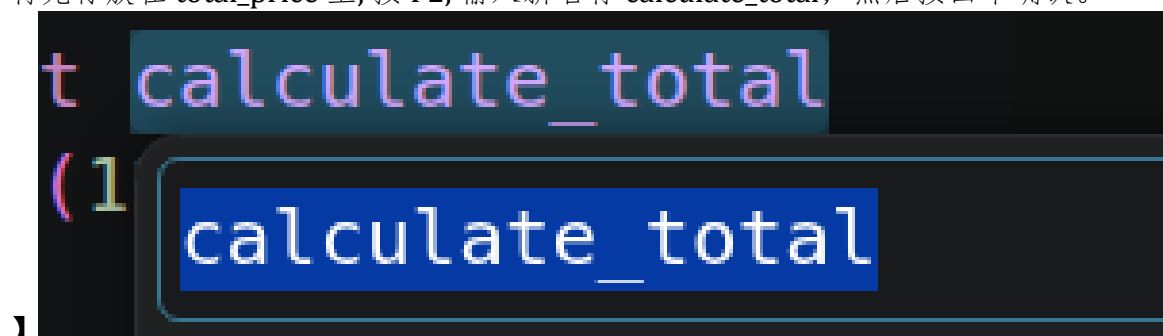


```
def total_price(price: float, count: int) -> float:
    return
```

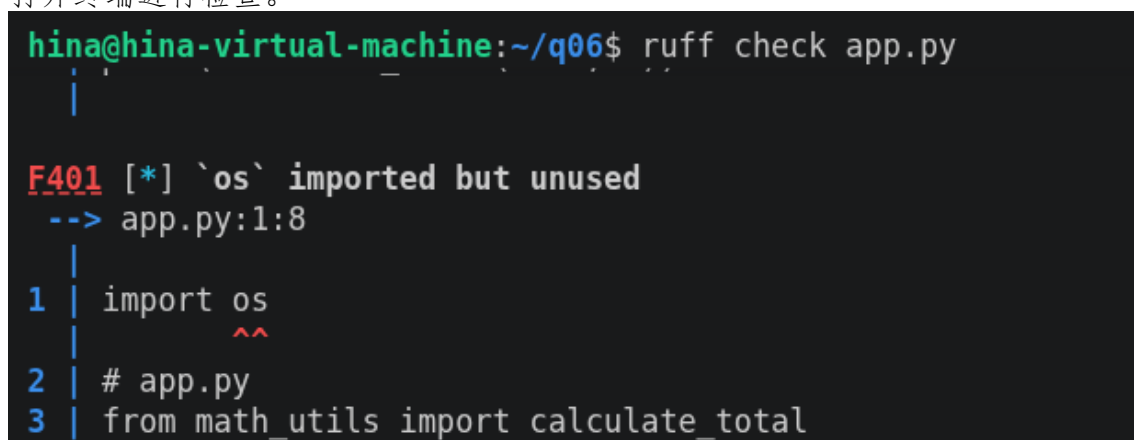
Go to References Shift+F12



将光标放在 `total_price` 上, 按 F2, 输入新名称 `calculate_total`, 然后按回车确认。



添加未使用的 `import os` 并用 `ruff` 检查: 在 `app.py` 顶部手动加入 `import os`, 保存, 打开终端进行检查。



删除刚才添加的 `import os` 行，保存文件，再次运行 `ruff check app.py`，此时通过检查。

```
Problems  Output  Debug Console  Terminal  Ports
● hina@hina-virtual-machine:~/-/q06$ ruff check app.py
All checks passed!
○ hina@hina-virtual-machine:~/-/q06$
```

2.3 踩坑记录

1. 写报告的时候 “_” 记得要使用转义符号。
2. `from...import...` 之后要换一行，否则检查不通过

3 第 7 题：用调试器定位归并排序缺陷

3.1 题目要求

在 `q07` 目录下完成以下操作：

1. 先运行给定输入，确认结果错误或出现异常。
2. 使用 `pdb`、`IDEdebugger` 或等价调试器，在 `merge` 中设置断点并观察 `i`、`j`、`left[i]`、`right[j]`。
3. 完成最小修复，不得用 `sorted` 替换归并排序。
4. 使用 `pytest` 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

3.2 实现过程

运行给定输入，确认结果错误或出现异常。

```
/usr/bin/python3 /home/hina/-/q07/merge_sort.py
● hina@hina-virtual-machine:~$ /usr/bin/python3 /home/hina/-/q07/merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
○ hina@hina-virtual-machine:~$
```

在第 5 行左侧单击，出现红点（断点）。启动调试，按 `F5`，选择 `Python File`，程序会在断点处暂停。观察变量，在左侧“变量”面板查看 `i`、`j`、`left[i]`、`right[j]` 的值。分析：

```
1 return merge(merge\_sort([3,1,4,1]),merge\_sort([5,9,2,6]))\\
```



```

2 #等价 merge(merge(merge([3],[1]),merge([4],[1])),merge(merge
3 ([5],[9]),merge([2],[6])))\\
#等价 merge(merge([1,3],[1,4])……)\\

```

Listing 2. 分析

第 1 次: $\text{left}[0]=1 \leq \text{right}[0]=1$ True $\rightarrow \text{result.append}(1), i=1$

第 2 次: $i=1, j=0, \text{left}[1]=3 \leq \text{right}[0]=1$? False $\rightarrow$ else: $\text{result.append}(\text{right}[i]) \rightarrow \text{right}[1]$ 是 4, 但正确的应该是 $\text{right}[0]=1$ 。错误发生

第 3 次: $i=1, j=1$, while $i < 2$ and $j < 2$? $i=1 < 2$ and $j=1 < 2$ True $\rightarrow \text{left}[1]=3 \leq \text{right}[1]=4$ True $\rightarrow \text{result.append}(3), i=2$

退出循环, 返回 $[1,4,3,4]$, 但正确排序应为 $[1,1,3,4]$ 。明显错误。

```

home > hina > - > q07 > merge_sort.py > merge_sort
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i])
7             i += 1
8         else:
9             result.append(right[i])
10            j += 1
11            return result + left[i:] + right[j:]
12
13 def merge_sort(a):
14     if len(a) <= 1:
15         return a
16     m = len(a) // 2
17     return merge(merge_sort(a[:m]), merge_sort(a[m:])) a = [3, 1, 4, 1], m = 2

```

当 i 和 j 不同步时, 会导致取错右序列的元素, 最终破坏排序的稳定性并产生错误顺序。

合并 $[1,3]$ 和 $[1,4]$ 时发现 $i=1, j=0$, 此时 $\text{right}[i]$ 取的是第二个元素 (4), 而正确应取 $\text{right}[j]$ (元素 1), 定位到错误。

因此, 将 else 下的 $\text{result.append}(\text{right}[i])$ 中的 i 改为 j 。

```

Problems Output Debug Console Terminal Ports
/usr/bin/python3 /home/hina/-/q07/merge_sort.py
hina@hina-virtual-machine:~$ /usr/bin/python3 /home/hina/-/q07/merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
hina@hina-virtual-machine:~$ /usr/bin/python3 /home/hina/-/q07/merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
hina@hina-virtual-machine:~$

```

使用 `pytest` 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过。

```

Problems Output Debug Console Terminal Ports
hina@hina-virtual-machine:~/q07$ python3 -m pytest -v
cachedir: .pytest_cache
rootdir: /home/hina/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED
test_merge_sort.py::test_duplicates PASSED

===== 2 passed in 0.01s =====

```

3.3 踩坑记录

1. 使用 `pytest` 时，前面指令应是 `python3`。

4 第 8 题：先测量，再优化慢速词频程序

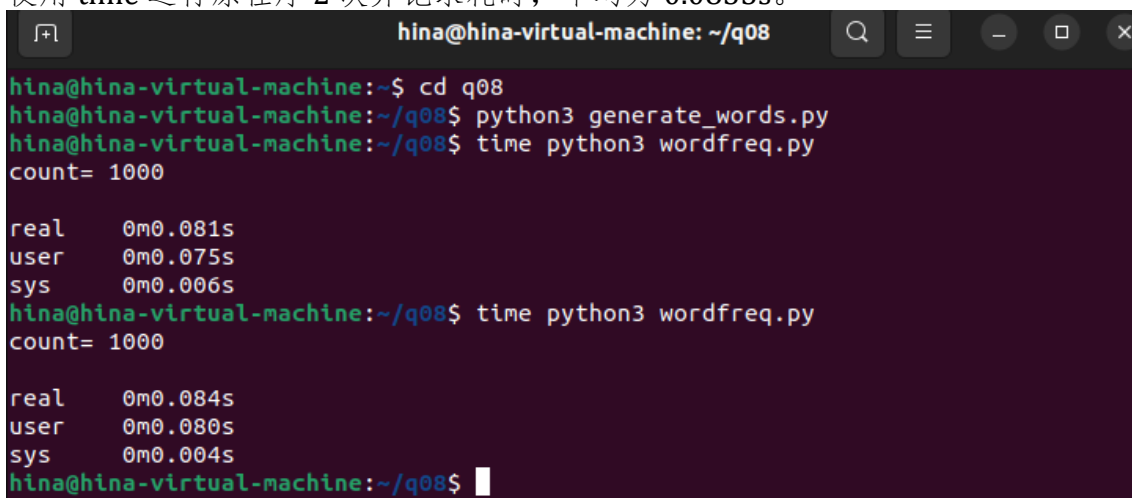
4.1 题目要求

在 `q08` 目录下完成以下操作：

1. 生成 `words.txt`，使用 `time` 运行原程序 2 次并记录耗时。
2. 使用 `cProfile-s cumulative` 运行一次，找出主要耗时位置。
3. 在不改变最终 `count` 的前提下优化程序；不得写死 1000。
4. 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

4.2 实现过程

使用 `time` 运行原程序 2 次并记录耗时，平均为 0.0835s。



```
hina@hina-virtual-machine: ~/q08
hina@hina-virtual-machine:~$ cd q08
hina@hina-virtual-machine:~/q08$ python3 generate_words.py
hina@hina-virtual-machine:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.081s
user    0m0.075s
sys     0m0.006s
hina@hina-virtual-machine:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.084s
user    0m0.080s
sys     0m0.004s
hina@hina-virtual-machine:~/q08$
```

使用 `cProfile-s cumulative` 运行一次，找出主要耗时位置。

```

hina@hina-virtual-machine:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
      1012 function calls in 0.069 seconds

Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     1      0.000    0.000    0.069    0.069 {built-in method builtins.exec}
     1      0.069    0.069    0.069    0.069 wordfreq.py:1(<module>)
     1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
     1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrap
er' objects}
    1000      0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
}
     1      0.000    0.000    0.000    0.000 codecs.py:319(decode)
     1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_dec
ode}
     1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
     1      0.000    0.000    0.000    0.000 {built-in method io.open}
     1      0.000    0.000    0.000    0.000 codecs.py:309(__init__)
     1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
     1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Prof
iler' objects}
     1      0.000    0.000    0.000    0.000 codecs.py:260(__init__)

```

wordfreq.py 的累积耗时是 0.069s, read 和 split 消耗 0.001 秒, 非常快。

时间主要消耗在 for word in words: 循环里的 if word not in unique, 因为每次都要从头到尾扫描这个列表, 是 $O(n^2)$ 。

因此将此处改进即可解决问题。列表换成集合, 集合的 in 查找是 $O(1)$ 。

```

1 words = open("words.txt", encoding="UTF-8").read().split()
2 unique = set(words)
3 print("count", len(unique))

```

Listing 3. 提速后的代码

用时变短, 平均 0.0105s, 速度提升约 7.95 倍。

```

hina@hina-virtual-machine: ~/q08
hina@hina-virtual-machine:~/q08$ cd q08
hina@hina-virtual-machine:~/q08$ time python3 new.py
count 1000

real    0m0.010s
user    0m0.007s
sys     0m0.003s
hina@hina-virtual-machine:~/q08$ time python3 new.py
count 1000

real    0m0.011s
user    0m0.007s
sys     0m0.003s
hina@hina-virtual-machine:~/q08$

```

5 课后题

以下为课后练习题目，来自课程教学网站的课后习题以及代码实例。

5.1 第 1 题

题目内容

1. 写出一个 `ls` 命令，让它按下面的方式列出文件：
2. 包含所有文件，包括隐藏文件
3. 文件大小以易读格式显示（例如 454M）
4. 按最近修改时间排序，输出带颜色

实现过程

-a: 显示所有文件。
-l: 显示权限、链接数、所有者、组、大小、修改时间等。
-h: 将文件大小转换为易读格式。
-t: 按最近修改时间排序。
-color=auto: 输出带颜色。

```
1 ls -alht --color=auto
```

Listing 4. 使用到的代码

截图如下

```
hina@hina-virtual-machine:~$ ls -alht --color=auto
total 192K
-rw----- 1 hina hina 14K Sep  4 08:26 .bash_history
drwxr-x--- 24 hina hina 4.0K Sep  3 17:06 .
-rw-rw-r-- 1 hina hina  0 Sep  3 17:06 q06
drwxrwxr-x 19 hina hina 4.0K Aug 31 16:22 -
drwxr-xr-x 2 hina hina 4.0K Aug 31 16:07 Downloads
drwx----- 5 hina hina 4.0K Aug 31 15:03 .local
drwx----- 17 hina hina 4.0K Aug 31 15:03 .cache
drwx----- 6 hina hina 4.0K Aug 31 14:04 snap
drwxrwxr-x 4 hina hina 4.0K Aug 31 13:55 .vscode
drwx----- 3 hina hina 4.0K Aug 31 13:55 .copilot
drwxrwxr-x 3 hina hina 4.0K Aug 31 13:55 .vscode-shared
drwxrwxr-x 3 hina hina 4.0K Aug 31 13:52 .dotnet
drwx----- 20 hina hina 4.0K Aug 31 13:50 .config
-rw-rw-r-- 1 hina hina 640 Aug 31 13:45 packages.microsoft.gpg
-rw-rw-r-- 1 hina hina 177 Aug 31 13:45 .wget-hsts
-rw----- 1 hina hina 306 Aug 31 13:42 .pam_environment
-rwxrwxr-x 1 hina hina 232 Aug 29 10:20 f.sh
-rw----- 1 hina hina 6.7K Aug 28 16:19 .viminfo
-rw-rw-r-- 1 hina hina 10 Aug 28 16:16 document_26-08-28.txt
drwx----- 2 hina hina 4.0K Aug 25 10:16 .ssh
-rw-rw-r-- 1 hina hina 57 Aug 25 09:33 .gitconfig
-rw----- 1 hina hina 20 Aug 24 16:06 .lessht
drwxrwxr-x 2 hina hina 4.0K Oct 29 2025 fbnq
-rw----- 1 hina hina 42 Oct 24 2025 .python_history
drwxrwxr-x 3 hina hina 4.0K Oct 22 2025 go
drwxrwxr-x 3 hina hina 4.0K Oct 22 2025 projects
-rw-r--r-- 1 hina hina 3.8K Oct 22 2025 .bashrc
-rw-r--r-- 1 hina hina 3.8K Oct 22 2025 .bashrc.save
drwxrwxr-x 3 hina hina 4.0K Oct 22 2025 go1.25.3.linux-amd64
-rw----- 1 hina hina 12K Sep 30 2025 .hello.c.swp
-rw----- 1 hina hina 12K Sep 30 2025 .test00.swp
-rw-r--r-- 1 hina hina  0 Sep 30 2025 .sudo_as_admin_successful
-rw-r--r-- 1 hina hina 220 Sep 29 2025 .bash_logout
-rw-r--r-- 1 hina hina 807 Sep 29 2025 .profile
```

5.2 第 2 题

题目内容

假设你有一个很少失败的命令。为了调试它，你需要把它的输出保存下来，但等到一次失败运行可能会很耗时。

1. 写一个 **bash** 脚本，不断运行下面这个脚本直到它失败为止
2. 把标准输出和标准错误分别保存到文件里
3. 把结果打印出来

实现过程

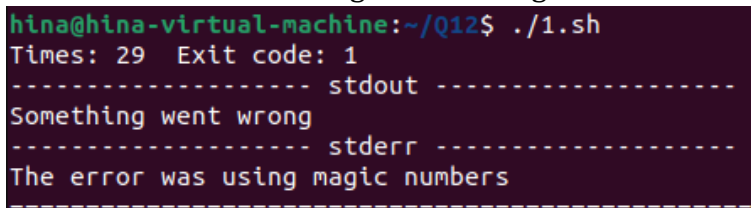
```

1  CMD="./test.sh"
2
3  TMP_OUT="stdout.tmp"
4  TMP_ERR="stderr.tmp"
5  FINAL_OUT="stdout.log"
6  FINAL_ERR="stderr.log"
7
8  count=0
9
10 while true; do
11     ((count++))
12     $CMD > "$TMP_OUT" 2> "$TMP_ERR"
13     exit_code=$?
14
15     if [ $exit_code -ne 0 ]; then
16         mv "$TMP_OUT" "$FINAL_OUT"
17         mv "$TMP_ERR" "$FINAL_ERR"
18
19         echo "Times: $count 次后失败 Exit Coed: $exit_code"
20         echo "----- stdout -----"
21         cat "$FINAL_OUT"
22         echo "----- stderr -----"
23         cat "$FINAL_ERR"
24         echo "===== "
25         break
26     fi
27 done

```

Listing 5. 使用到的代码

截图如下，同时 stderr.log 和 stdout.log 中均有对应内容



```

hina@hina-virtual-machine:~/Q12$ ./1.sh
Times: 29 Exit code: 1
----- stdout -----
Something went wrong
----- stderr -----
The error was using magic numbers
=====

```

5.3 第3题

题目内容

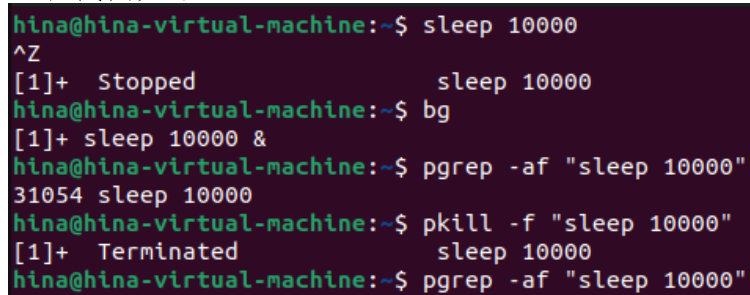
1. 终端里启动一个 `sleep 10000` 任务，用 `Ctrl-Z` 把它挂起，再用 `bg` 让它继续在后台运行。
2. 使用 `pgrep` 找到它的 `pid`，再用 `kill` 把它杀掉。

实现过程

```
1 sleep 10000
2 #按下 Ctrl+Z, 进程被挂起。
3 bg
4 pgrep -af "sleep 10000"
5 pkill -f "sleep 10000"
6 #确认进程杀掉
7 pgrep -af "sleep 10000"
```

Listing 6. 使用到的代码

过程截图如下



```
hina@hina-virtual-machine:~$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000
hina@hina-virtual-machine:~$ bg
[1]+ sleep 10000 &
hina@hina-virtual-machine:~$ pgrep -af "sleep 10000"
31054 sleep 10000
hina@hina-virtual-machine:~$ pkill -f "sleep 10000"
[1]+  Terminated              sleep 10000
hina@hina-virtual-machine:~$ pgrep -af "sleep 10000"
```

5.4 第4题

题目内容

1. - 是一个特殊参数-后面的所有内容都会被当作位置参数。
2. 试着运行 `touch - -myfile`，然后在不使用 - 的情况下把它删掉。

实现过程

```
1 touch -- -myfile
2 ls -l #确认文件存在
3 rm ./-myfile
```

Listing 7. 使用到的代码

截图如下

```

hina@hina-virtual-machine: ~
hina@hina-virtual-machine:~$ touch -- -myfile
hina@hina-virtual-machine:~$ ls -l
total 68
drwxrwxr-x 20 hina hina 4096 Sep  4 08:55 -
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Desktop
-rw-rw-r--  1 hina hina  10 Aug 28 16:16 document_26-08-28.txt
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Documents
drwxr-xr-x  2 hina hina 4096 Sep  4 08:46 Downloads
drwxrwxr-x  2 hina hina 4096 Oct 29 2025 fbnq
-rwxrwxr-x  1 hina hina 232 Aug 29 10:20 f.sh
drwxrwxr-x  3 hina hina 4096 Oct 22 2025 go
drwxrwxr-x  3 hina hina 4096 Oct 22 2025 go1.25.3.linux-amd64
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Music
-rw-rw-r--  1 hina hina  0 Sep  5 07:43 -myfile
-rw-rw-r--  1 hina hina 640 Aug 31 13:45 packages.microsoft.gpg
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Pictures
drwxrwxr-x  3 hina hina 4096 Oct 22 2025 projects
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Public
-rw-rw-r--  1 hina hina  0 Sep  3 17:06 q06
drwx----- 6 hina hina 4096 Aug 31 14:04 snap
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Templates
drwxr-xr-x  2 hina hina 4096 Sep 29 2025 Videos
hina@hina-virtual-machine:~$ rm -myfile
rm: invalid option -- 'm'
Try 'rm ./-myfile' to remove the file '-myfile'.
Try 'rm --help' for more information.
hina@hina-virtual-machine:~$ rm ./-myfile

```

5.5 第 5 题

题目内容

1. 试着配合 `diff` 和进程替换，比较 `printenv` 与 `export` 的输出。它们为什么不一样？

实现过程

```
1 diff <(<printenv | sort) <(<export | sort))
```

Listing 8. 使用到的代码

截图如下

`printenv` 和 `export` 输出不一样，是因为输出格式不同：`printenv` 用于机器解析，`export` 用于 shell 重新导入，因此包含声明关键字和引用处理。

```

hina@hina-virtual-machine: ~
hina@hina-virtual-machine:~$ diff <(<printenv | sort) <(<export | sort)
1,45c1,45
< COLORTERM=truecolor
< DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
< DESKTOP_SESSION=ubuntu
< DISPLAY=:0
< GDMSESSION=ubuntu
< GNOME_DESKTOP_SESSION_ID=this-is-deprecated
< GNOME_SETUP_DISPLAY=:1
< GNOME_SHELL_SESSION_MODE=ubuntu
< GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/f70268df_57b8_43aa_898c_232a2
eeaaab8
< GNOME_TERMINAL_SERVICE=:1.426
< GOPATH=/home/hina/go
< GTK_MODULES=gail:atk-bridge
< HOME=/home/hina
< IM_CONFIG_PHASE=1
< LANG=en_US.UTF-8
< LESSCLOSE=/usr/bin/lesspipe %s %s
< LESSOPEN=| /usr/bin/lesspipe %s
< LOGNAME=hina
< LS_COLORS=rs=0:di=01:34:ln=01:36:mh=00:pi=40:33:so=01:35:do=01:35:bd=40:33:01:

```



```

> declare -x OLDPWD
> declare -x PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/local/go/bin:/home/hina/go/bin"
> declare -x PWD="/home/hina"
> declare -x QT_ACCESSIBILITY="1"
> declare -x QT_IM_MODULE="ibus"
> declare -x SESSION_MANAGER="local/hina-virtual-machine:@/tmp/.ICE-unix/1264,unix/hina-virtual-machine:/tmp/.ICE-unix/1264"
> declare -x SHELL="/bin/bash"
> declare -x SHLVL="1"
> declare -x SSH_AGENT_LAUNCHER="gnome-keyring"
> declare -x SSH_AUTH_SOCK="/run/user/1000/keyring/ssh"
> declare -x SYSTEMD_EXEC_PID="30128"
> declare -x TERM="xterm-256color"
> declare -x USER="hina"
> declare -x USERNAME="hina"
> declare -x VTE_VERSION="6800"
> declare -x WAYLAND_DISPLAY="wayland-0"
> declare -x XAUTHORITY="/run/user/1000/.mutter-Xwaylandauth.8SR5U3"
> declare -x XDG_CONFIG_DIRS="/etc/xdg/xdg-ubuntu:/etc/xdg"
> declare -x XDG_CURRENT_DESKTOP="ubuntu:GNOME"
> declare -x XDG_DATA_DIRS="/usr/share/ubuntu:/usr/local/share:/usr/share:/var/lib/snapd/desktop"
> declare -x XDG_MENU_PREFIX="gnome-"
> declare -x XDG_RUNTIME_DIR="/run/user/1000"
> declare -x XDG_SESSION_CLASS="user"
> declare -x XDG_SESSION_DESKTOP="ubuntu"
> declare -x XDG_SESSION_TYPE="wayland"
> declare -x XMODIFIERS="@im=ibus"

```

5.6 第 6 题

题目内容

1. 写两个 `bash` 函数 `marco` 和 `polo`
2. 每次执行 `marco` 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 `polo`，它都应该把你 `cd` 回执行 `marco` 时所在的目录。
3. 为了方便调试，你可以把代码写进 `marco.sh`，然后通过执行 `source marco.sh` 把这些定义重新加载到当前 `shell`。

实现过程

```

1  #!/bin/bash
2
3  # 保存当前工作目录到变量 MARCO_DIR
4  marco() {
5      export MARCO_DIR="$(pwd)"
6      echo "Saved directory: $MARCO_DIR"
7  }
8
9  # 切换回保存的目录
10 polo() {
11     if [ -n "$MARCO_DIR" ]; then
12         cd "$MARCO_DIR" || echo "Failed to cd to $MARCO_DIR"
13         echo "Now in: $(pwd)"
14     else
15         echo "Error: No directory saved. Run 'marco' first."
16     fi

```

17 }

Listing 9. 使用到的代码

截图如下，可以看到在 Q16 文件夹使用 marco 后切换目录使用 polo 就切换回去了。

```
hina@hina-virtual-machine: ~/Q16
hina@hina-virtual-machine:~$ cd Q16
hina@hina-virtual-machine:~/Q16$ source marco.sh
hina@hina-virtual-machine:~/Q16$ pwd
/home/hina/Q16
hina@hina-virtual-machine:~/Q16$ marco
Saved directory: /home/hina/Q16
hina@hina-virtual-machine:~/Q16$ cd ~/-
hina@hina-virtual-machine:~/$ pwd
/home/hina/-
hina@hina-virtual-machine:~/$ polo
Now in: /home/hina/Q16
hina@hina-virtual-machine:~/Q16$ pwd
/home/hina/Q16
hina@hina-virtual-machine:~/Q16$
```

5.7 第 7 题

题目内容

1. 你不想在某个进程完成之前启动另一个进程，限制条件始终是 `sleep 60 &`。
2. 一种实现方式是使用 `wait` 命令。试着启动这个 `sleep`，然后让一个 `ls` 等到后台进程结束后再执行。

实现过程

```
1 sleep 60 &           # 启动后台任务
2 pid=$!              # 获取该任务的进程 ID
3 wait $pid           # 只等待这个特定进程结束
4 ls
```

Listing 10. 使用到的代码

截图如下，可以看出输入的 `ls` 在 `sleep` 结束之后被执行

```
hina@hina-virtual-machine:~$ sleep 60 &
[1] 40468
hina@hina-virtual-machine:~$ pid=$!
hina@hina-virtual-machine:~$ wait $pid
ls
[1]+  Done                  sleep 60
hina@hina-virtual-machine:~$ ls
- Desktop Downloads go1.25.3.linux-amd64 projects Templates
document_26-08-28.txt fbng Music Public Videos
Documents go packages.microsoft.gpg q06
Pictures snap
```

5.8 第 8 题

题目内容

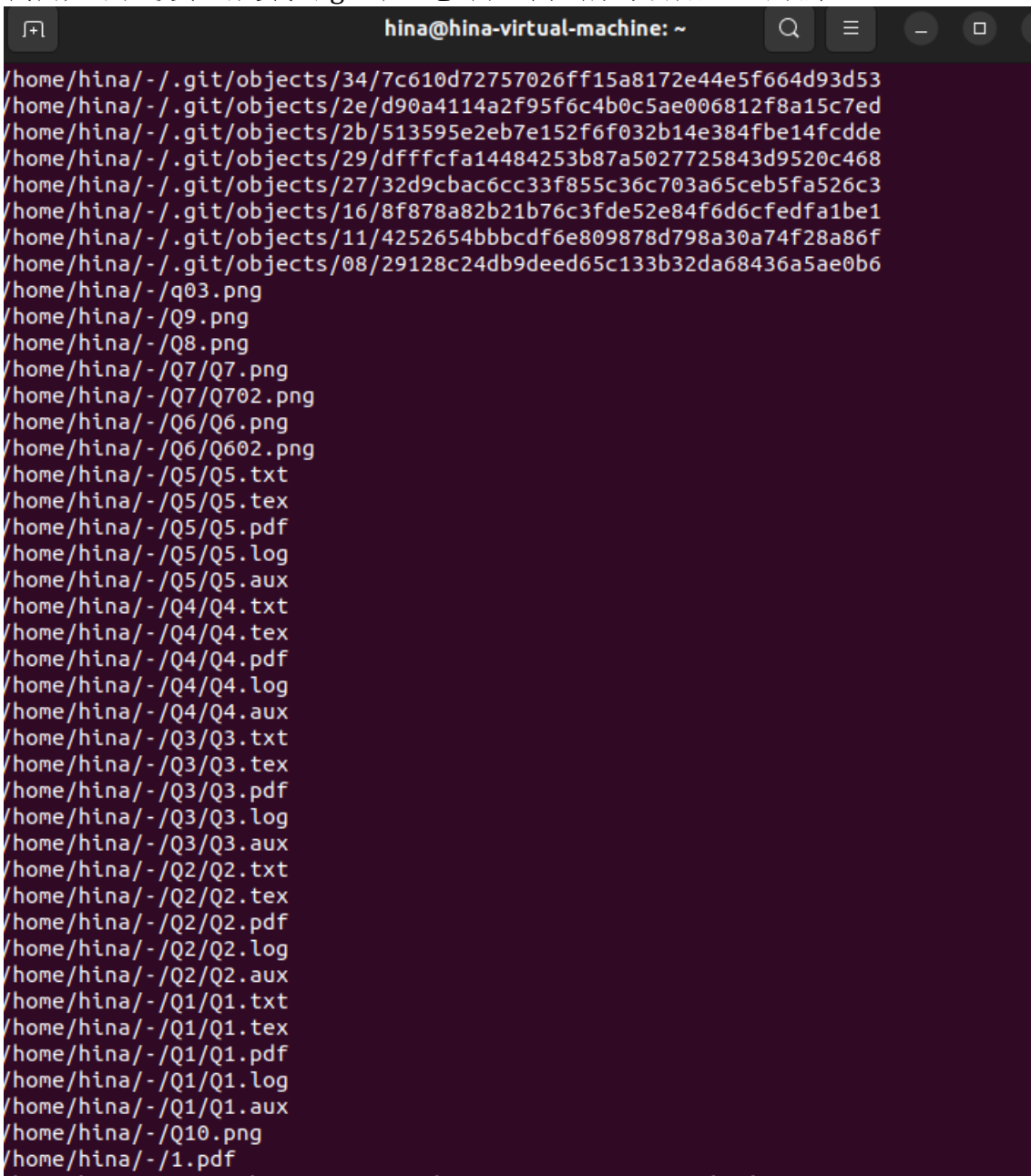
1. 按“最近修改时间”列出所有文件

实现过程

```
1 find ~/- -type f -printf '%T@ %p\n' | sort -rn | cut -d' ' -f2-
```

Listing 11. 使用到的代码

截图如下，搜索的是我的 git 本地仓库，可以看见文件加入的顺序



```
hina@hina-virtual-machine: ~  
/home/hina/-/.git/objects/34/7c610d72757026ff15a8172e44e5f664d93d53  
/home/hina/-/.git/objects/2e/d90a4114a2f95f6c4b0c5ae006812f8a15c7ed  
/home/hina/-/.git/objects/2b/513595e2eb7e152f6f032b14e384fbe14fcdde  
/home/hina/-/.git/objects/29/dfffcfa14484253b87a5027725843d9520c468  
/home/hina/-/.git/objects/27/32d9cbac6cc33f855c36c703a65ceb5fa526c3  
/home/hina/-/.git/objects/16/8f878a82b21b76c3fde52e84f6d6cfedfa1be1  
/home/hina/-/.git/objects/11/4252654bbbcdf6e809878d798a30a74f28a86f  
/home/hina/-/.git/objects/08/29128c24db9deed65c133b32da68436a5ae0b6  
/home/hina/-/q03.png  
/home/hina/-/Q9.png  
/home/hina/-/Q8.png  
/home/hina/-/Q7/Q7.png  
/home/hina/-/Q7/Q702.png  
/home/hina/-/Q6/Q6.png  
/home/hina/-/Q6/Q602.png  
/home/hina/-/Q5/Q5.txt  
/home/hina/-/Q5/Q5.tex  
/home/hina/-/Q5/Q5.pdf  
/home/hina/-/Q5/Q5.log  
/home/hina/-/Q5/Q5.aux  
/home/hina/-/Q4/Q4.txt  
/home/hina/-/Q4/Q4.tex  
/home/hina/-/Q4/Q4.pdf  
/home/hina/-/Q4/Q4.log  
/home/hina/-/Q4/Q4.aux  
/home/hina/-/Q3/Q3.txt  
/home/hina/-/Q3/Q3.tex  
/home/hina/-/Q3/Q3.pdf  
/home/hina/-/Q3/Q3.log  
/home/hina/-/Q3/Q3.aux  
/home/hina/-/Q2/Q2.txt  
/home/hina/-/Q2/Q2.tex  
/home/hina/-/Q2/Q2.pdf  
/home/hina/-/Q2/Q2.log  
/home/hina/-/Q2/Q2.aux  
/home/hina/-/Q1/Q1.txt  
/home/hina/-/Q1/Q1.tex  
/home/hina/-/Q1/Q1.pdf  
/home/hina/-/Q1/Q1.log  
/home/hina/-/Q1/Q1.aux  
/home/hina/-/Q10.png  
/home/hina/-/1.pdf
```

5.9 第 9 题

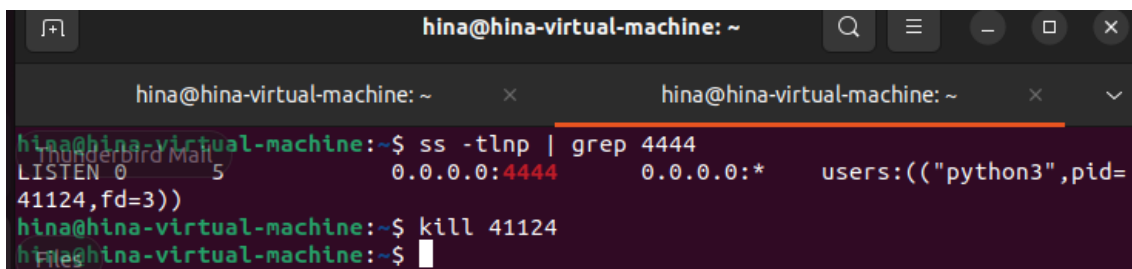
题目内容

1. 执行以启动一个最小的网络服务器，位于 4444 端口。在另一个终端上运行以查找进程。用来终止。

实现过程

```
1 #terminal 1
2 python -m http.server 4444
3 #terminal 2
4 ss -tlnp | grep 4444
5 kill 41124
```

Listing 12. 使用到的代码



```
hina@hina-virtual-machine: ~
hina@hina-virtual-machine: ~
hina@hina-virtual-machine:~$ ss -tlnp | grep 4444
LISTEN 0 5 0.0.0.0:4444 0.0.0.0:* users:((\"python3\",pid=
41124,fd=3))
hina@hina-virtual-machine:~$ kill 41124
hina@hina-virtual-machine:~$
```

5.10 第 10 题

题目内容

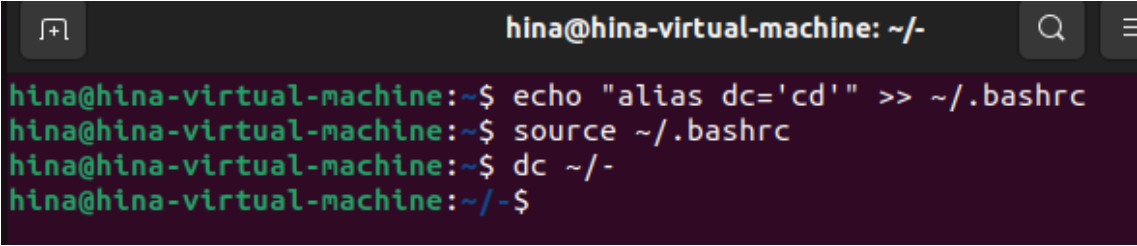
1. 创建一个 alias dc，让你把 cd 打错时也能正常工作。

实现过程

```
1 echo "alias dc='cd'" >> ~/.bashrc
2 source ~/.bashrc
```

Listing 13. 使用到的代码

每次打开终端都自动加载该别名，效果如下：



```
hina@hina-virtual-machine: ~/-  
hina@hina-virtual-machine:~$ echo "alias dc='cd'" >> ~/.bashrc  
hina@hina-virtual-machine:~$ source ~/.bashrc  
hina@hina-virtual-machine:~$ dc ~/-  
hina@hina-virtual-machine:~/- $
```

个人感想

通过本次实验，我对 **Linux** 命令行、调试器及性能分析工具的实际运用有了更直观的体会，尤其是在定位问题根源和进行针对性优化方面积累了经验。这些工具链的熟练使用，将为我今后的开发与调试工作提供扎实的支持。

总结

本次实验完成了四项独立任务及十道综合练习题，覆盖了以下核心技能：

1. **Shell 任务控制**：掌握 **bg**、**jobs**、**kill** 及 **PID** 获取，理解前台/后台/挂起状态转换；
2. **语言服务器应用**：使用 **VSCode + Python** 插件实现跳转、引用查找和全局重命名，熟悉 **Ruff** 静态检查；
3. **调试与缺陷修复**：利用断点调试定位归并排序中的索引错误，并通过 **pytest** 增加回归测试；
4. **性能分析与优化**：借助 **cProfile** 识别时间复杂度瓶颈，通过数据结构替换将词频统计速度提升约 8 倍；
5. **扩展 Shell 技巧**：包括别名、进程替换、**wait** 同步、**find** 排序、网络服务控制等，丰富了日常开发工具箱。

所有任务均基于 **Linux** 环境完成，并通过 **Git** 进行版本管理。本次实验不仅强化了基础工具链的使用，也培养了系统化排错和性能调优的工程素养。